

AI Critique Lab 3

Recommended Technology Stack (Entry-Level Friendly)

1. Frontend (Mobile App)

Option A (Recommended): React Native + Expo

- Language: TypeScript
- Framework: React Native (Expo)
- Navigation: expo-router / React Navigation
- UI: Native components + simple UI libraries

Why

- You already know React → minimal learning curve
- Cross-platform (Android + iOS)
- Fast for prototyping (important for lab timeline)

Key Libraries

- `react-native-maps` → map display (matches your MapController)
 - `axios` → API calls
 - `zustand` or simple React state → lightweight state management
-

2. Backend (Core Logic Layer)

Recommended: Java (Spring Boot)

- Framework: Spring Boot
- Architecture: REST API (fits your Control classes → Controllers)
- ORM: Spring Data JPA
- Build tool: Maven / Gradle

Why

- You already know Java
 - Maps directly to your UML:
 - Entity → JPA Entity
 - Control → Spring Controller/Service
 - Clean separation of logic (good for marks in Lab 3)
-

3. Database (Persistent Data)

Recommended: PostgreSQL

- Tables map directly to your entities:
 - User, Wallet, Transaction, Booking, EVStation, Carpark

Why

- Relational fits your structured data
 - Easy with Spring Boot (JPA)
 - Industry standard
-

4. API Layer (External Integrations)

Your system explicitly depends on external APIs :

Maps & Location

- Google Maps API OR Mapbox
- Nominatim (if you want free option)

EV / Carpark Data

- LTA DataMall (Singapore)
- Custom mock API if needed

Payment (Simple for project)

- Stripe (test mode) OR mock payment service
 - PayNow → simulate QR (no need real integration)
-

5. Authentication

Recommended: JWT-based authentication

- Spring Security + JWT
- Flow:
 - Login → token
 - Token used for API requests

Why

- Simple + industry standard
 - Matches your login/register flows
-

6. Deployment (Keep it simple)

- Backend: Render / Railway / Fly.io
 - Database: Supabase / Railway Postgres
 - Frontend: Expo (no deployment headache)
-

7. Development Tools

- GitHub (required by lab)
 - Postman (API testing)
 - VSCode / IntelliJ
-



How This Maps to Your UML (Important for marks)

Your BCE design maps cleanly:

Boundary → Frontend (React Native)

- MapInterface → Screens
- BookingInterface → UI components
- WalletInterface → UI

Control → Spring Boot

- MapController → REST endpoint
- BookingController → Service + Controller
- WalletController → Service

Entity → Database

- Wallet, Transaction, Booking → DB tables

This alignment is exactly what Lab 3 expects .



What NOT to use (overkill for your project)

Avoid:

- Microservices
- Kubernetes
- GraphQL (REST is enough)

- Redux (too heavy for your scale)
-

Final Stack Summary

Frontend

- React Native (Expo) + TypeScript

Backend

- Java Spring Boot (REST API)

Database

- PostgreSQL

APIs

- Google Maps / Mapbox
- LTA DataMall (or mock)

Auth

- JWT

Deployment

- Render / Railway + Expo
-

Short justification (for your report)

This stack is suitable because it aligns with the team's existing experience in Java and React, reduces learning overhead, and supports rapid development of a mobile application with real-time features such as maps, booking, and payments. It also maps cleanly to the BCE architecture, improving maintainability and traceability from design to implementation.